5-1 · 분산 처리와 Spark 아키텍처 · 실습 1H

Spark UI로 잡 구조 읽기

Job → Stage → Task 와 DAG, 셔플이 Stage 를 가른다

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

이 시간의 목표

1시간 뒤에는

① Jobs 탭에서 Job 이 action 수보다 많은 것을 본다

② groupBy 의 Job 둘과 explain 의 Exchange 를 찾는다

③ 계획의 200 과 실제 Task 1 — AQE 가 합친 것을 확인한다

④ cache 시간을 재 보고 Storage 탭에서 캐시를 확인한다

Executors 탭에 driver 하나뿐인 화면을 기억해 두세요 — 5-2 에서 워커로 채워집니다

실습 흐름

일곱 단계 — 탭을 하나씩 열며 계층을 읽는다

앞 실습의 셸과 localhost:4040 그대로 — 탭 여섯(SQL / DataFrame 포함)

1. Jobs 탭	2·3. 셔플·Task	4·5. DAG·실행자	6. cache	7. 실패·나가기
Job 이 action 수보다 많다	groupBy Job 둘 200 → Task 1	skipped 박스 driver 하나	count 세 번 Storage 탭	4040 겹침 exit()

1단계

Jobs 탭 — action 하나가 Job 여럿

열 · 항목	뜻
Description	count at … · showString at …
Duration	그 Job 이 걸린 시간
Stages 1/1	단계 하나를 실행
1/1 (1 skipped)	앞 Job 이 해 둔 단계를 건너뜀
Job 클릭	Stage 목록 + DAG Visualization

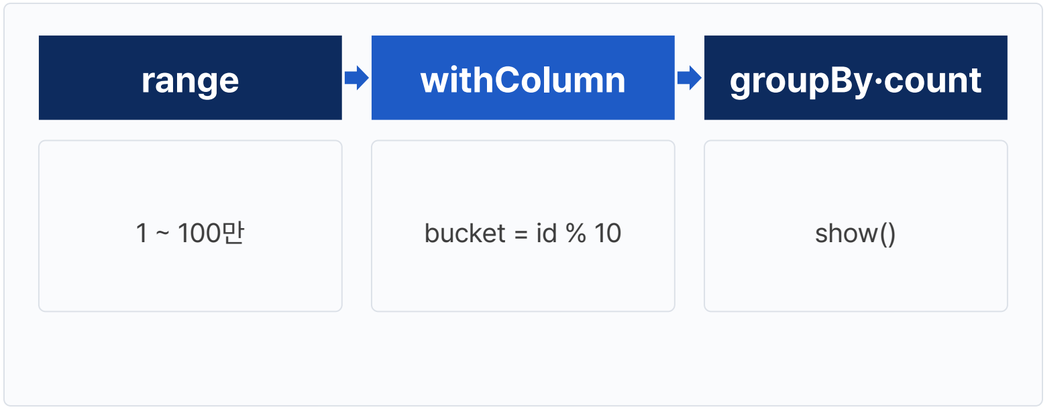
확인: count() 가 Job 0 · 1, show(5) 가 Job 2 · 3 · 4 — 파티션별 개수 세기까지 일곱 줄 안팎

2단계

groupBy 로 셔플을 만들면 Job 이 둘 늘어난다

코드 — 같은 키를 한곳에

결과 bucket 0~9 가 100000 씩(순서는 섞임)



VS

Jobs 탭과 explain()

「나누기」와 「모아서 세기」 사이가 셔플

- Job 7 — Stage 하나, Task 16
- Job 8 — 1/1 (1 skipped), Task 1
- Exchange hashpartitioning(..., 200)
- Exchange = 셔플 · 맨 위 AdaptiveSparkPlan

3단계

Stages 탭 — 계획은 200, 실제 Task 는 1

1. **Stage 9 (Job 7) — Task 16**
   파티션 수와 같다 · 파티션마다 62500 행을 읽어 나눈다
2. **Stage 11 (Job 8) — Task 1**
   계획은 200(셔플 파티션 기본값) — AQE 가 실행 중에 합쳤다
3. **Duration 열**
   한 Task 만 유난히 오래 걸리면 데이터가 한쪽에 몰린 것(스큐)
4. **Summary Metrics 의 Min · Median · Max**
   비교하면 치우침이 한눈에 보인다

요점

계획과 실제는 다를 수 있다

4·5단계

DAG 를 읽는다 — 박스가 Stage, 선이 셔플

Job 7	Job 8 · Stage 10	사이의 선	Stage 11
Stage 9 하나 끝이 Exchange	skipped · 회색 끝에 Exchange	셔플	AQEShuffleRead 로 시작

박스 안 상자가 연산 · 위에서 아래로 읽기 — 이론 챕터의 도해와 같은 구조

5단계 Executors 탭: driver 하나 · Cores 16(Docker 에 준 CPU 수 — 사람마다 다름) — 5-2 에서 워커가 줄줄이

6단계

cache — 시간 차이는 작다, Storage 탭을 본다

확인	실측
count 세 번 (1회)	0.27 · 0.19 · 0.17초
count 세 번 (2회)	0.57 · 0.33 · 0.38초 (역전)
Cached Partitions	16 (파티션 수)
Fraction Cached	100%
Size in Memory	2.8 MiB
다 보면	big.unpersist()

spark.range 는 숫자를 그 자리에서 만들어 캐시 읽기와 비용이 비슷 — 캐시는 정말 여러 번 쓸 때만

7단계 — 일부러 실패시키기

셸을 하나 더 띄우면 4040 이 겹친다

셸을 띄운 채 새 창에서 같은 docker run → Bind for 0.0.0.0:4040 failed: port is already allocated

새 창에서	실패	이유	해결	exit() 뒤
같은 docker run -p 4040:4040	셸이 안 뜨고 바로 끝난다	4040 을 앞 컨테이너가 잡음	앞 셸 exit() 또는 -p 4041:4040	docker ps -a 제목 줄만

확인 포인트

이 1시간이 끝날 때 확인할 것

1. Jobs 탭에 1/1 과 1/1 (1 skipped) 줄이 섞여 있다
2. groupBy 가 Job 둘 — Task 16 / skipped + Task 1
3. explain 에 Exchange …(…, 200), 실제 Task 는 1
4. Job 8 DAG 에 회색 skipped 박스와 AQEShuffleRead
5. Executors 탭에 driver 하나, Storage 탭에 16 · 2.8 MiB
6. exit() 뒤 docker ps -a 는 제목 줄만, 4040 닫힘

5-1 · 실습 1H 마무리

다음은 정리·회고 (0.5H)

오늘 본 개념 아홉 개를 표로 정리하고, 5-2 클러스터를 위한 이미지·메모리를 확인합니다

- ✓ 개념 9개 — 한 줄 정의와 UI 위치
- ✓ docker images apache/spark (1.56GB)
- ✓ docker info — Total Memory 4GiB 이상
- ✓ 체크리스트 · 자주 겪는 오류

가져갈 것

cache 세 시간 · Cores 수 메모